Segment Tree

Arman Ferdous

Segment Tree

- A segment tree is a binary tree that contains the result of specific intervals of the array.
- Root node contains the result (in this case: sum of values) of all indices.
- If a node contains the information for the range [L, R] where L < R, then it has two children. Let mid = (L + R) / 2.
  Left child: [L, mid]
  Right child: [mid + 1, R]

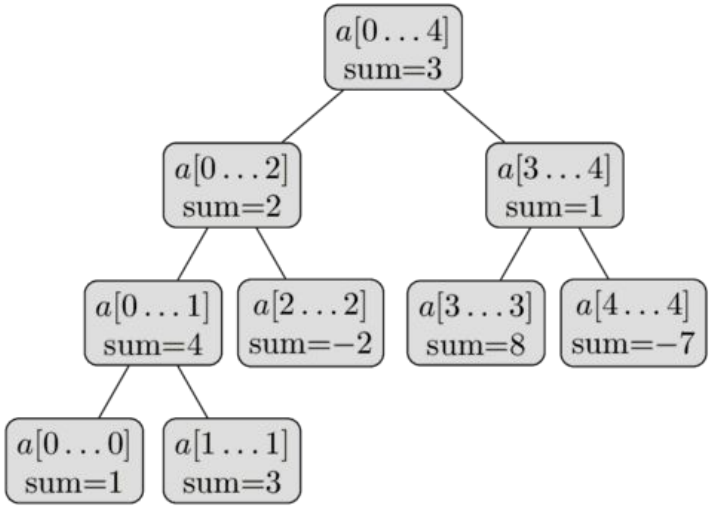


Image: cp-Algorithms

Point Update Range Sum Problem

- Given an array of n integers, handle Q queries of type:

  1 i x: set a[i] = x
  2 l r: output a[l] + a[l+1] + … + a[r]

- We already know how to solve the problem without any updates using prefix sum technique. Now we will see how to handle updates.

Building a Segment Tree

```
int a[N];
int t[MAX];

void build (int u, int b, int e) {
    if (b == e) {
        t[u] = a[b];
        return;
    }
    int mid = (b + e) / 2;
    build(2 * u, b, mid);
    build(2 * u + 1, mid + 1, e);

    t[u] = t[2 * u] + t[2 * u + 1];
}
```

Querying in a Segment Tree

```
int query (int u, int b, int e, int l, int r) {
    if (r < b || e < l) return 0;
    if (l <= b && e <= r) return t[u];

    int mid = (b + e) / 2;
    int lc = 2 * u, rc = 2 * u + 1;

    int left_contribution = query(lc, b, mid, l, r);
    int right_contribution = query(rc, mid + 1, e, l, r);

    return left_contribution + right_contribution;
}
```

Updating in a Segment Tree

```
void update (int u, int b, int e, int pos, int x) {
    if (b == e) {
        t[u] = x;
        //a[pos] = x;
        return;
    }
    int mid = (b + e) / 2;
    int lc = 2 * u, rc = 2 * u + 1;

    if (pos <= mid) update(lc, b, mid, pos, x);
    else update(rc, mid + 1, e, pos, x);

    t[u] = t[2 * u] + t[2 * u + 1];
}
```

Range Add Point Query Problem

- Given an array of n integers, handle Q queries of type:

  1 L R x: do a[L] += x, a[L+1] += x, …, a[R] += x
  2 i: output a[i]
- Main idea: Change what the node stores.

Range Addition Range Sum Problem

- Given an array of n integers, handle Q queries of type:

  1 L R x: do a[L] += x, …, a[R] += x
  2 L R: output a[L] + … + a[R]

Code: https://ideone.com/Tq9C86

Lazy Propagation

- Keep a lazy value to indicate the state of the subtree.
  - In this case: lazy = x implies every node in the subtree needs to add x to them.
- When updating, if you can leave a lazy tag on a node, do it immediately and don’t go down any further.
- If a node needs to go down from a node which has lazy value in it, **push** that lazy value first before you go down (otherwise you will get old values).
- Remember, any function that can push lazy down, must also merge two children when going back up.

Methods involved

- Build: To build a segment tree, provided the leaves.
- Merge: To combine the results of two children.
- Push: To push the lazy value downwards 1 level.
- Update: To perform range updates.
- Query: To perform range queries

https://cses.fi/problemset/task/1735

Time limit: 1.00 s **Memory limit:** 512 MB

Your task is to maintain an array of n values and efficiently process the following types of queries:

1. Increase each value in range $[a,b]$ by x.
2. Set each value in range $[a,b]$ to x.
3. Calculate the sum of values in range $[a,b]$.

Input

The first input line has two integers n and q: the array size and the number of queries.

The next line has n values $t_1,t_2,\ldots,t_n$: the initial contents of the array.

Finally, there are q lines describing the queries. The format of each line is one of the following: "1 a b x", "2 a b x", or "3 a b".

Output

Print the answer to each sum query.

Constraints

- $1 \le n,q \le 2 \cdot 10^5$
- $1 \le t_i,x \le 10^6$
- $1 \le a \le b \le n$

https://cses.fi/problemset/task/1734

You are given an array of n integers and q queries of the form: how many distinct values are there in a range $[a, b]$?

Input

The first input line has two integers n and q: the array size and number of queries.

The next line has n integers $x_1, x_2, \ldots, x_n$: the array values.

Finally, there are q lines describing the queries. Each line has two integers a and b.

Output

For each query, print the number of distinct values in the range.

Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $1 \le x_i \le 10^9$
- $1 \le a \le b \le n$

Code: https://ideone.com/tUGYM9

https://cses.fi/problemset/task/1736

Your task is to maintain an array of n values and efficiently process the following types of queries:

1. Increase the first value in range $[a, b]$ by 1, the second value by 2, the third value by 3, and so on.
2. Calculate the sum of values in range $[a, b]$.

Input

The first input line has two integers n and q: the size of the array and the number of queries.

The next line has n values $t_1, t_2, \ldots, t_n$: the initial contents of the array.

Finally, there are q lines describing the queries. The format of each line is either "1 a b" or "2 a b".

Output

Print the answer to each sum query.

Constraints

- $1 \le n, q \le 2 \cdot 10^5$
- $1 \le t_i \le 10^6$
- $1 \le a \le b \le n$

https://codeforces.com/problemset/problem/718/C

C. Sasha and Array

time limit per test: 5 seconds
memory limit per test: 256 megabytes

Sasha has an array of integers $a_1, a_2, ..., a_n$. You have to perform m queries. There might be queries of two types:

1. `1 l r x` — increase all integers on the segment from l to r by values x;
2. `2 l r` — find $\sum_{i=l}^{r} f(a_i)$, where $f(x)$ is the x-th Fibonacci number. As this number may be large, you only have to find it modulo $10^9 + 7$.

In this problem we define Fibonacci numbers as follows: $f(1) = 1, f(2) = 1, f(x) = f(x - 1) + f(x - 2)$ for all $x > 2$.

Sasha is a very talented boy and he managed to perform all queries in five seconds. Will you be able to write the program that performs as well as Sasha?

Input

The first line of the input contains two integers n and m ($1 \le n \le 100\,000$, $1 \le m \le 100\,000$) — the number of elements in the array and the number of queries respectively.

The next line contains n integers $a_1, a_2, ..., a_n$ ($1 \le a_i \le 10^9$).

Then follow m lines with queries descriptions. Each of them contains integers tp_i, l_i, r_i and may be x_i ($1 \le tp_i \le 2$, $1 \le l_i \le r_i \le n$, $1 \le x_i \le 10^9$). Here $tp_i = 1$ corresponds to the queries of the first type and tp_i corresponds to the queries of the second type.

It's guaranteed that the input will contains at least one query of the second type.

Output

For each query of the second type print the answer modulo $10^9 + 7$.

Implementations

- Sparse Table: https://github.com/Arman-Ferdous/kactl/blob/main/content/data-structures/RMQ.h
- Segment Tree (Lazy): https://github.com/Arman-Ferdous/kactl/blob/main/content/data-structures/SegmentTree.h
- Fenwick Tree: https://github.com/Arman-Ferdous/kactl/blob/main/content/data-structures/FenwickTree.h